Configuração do WordPress com Múltiplas Instâncias Utilizando Nginx e Docker Compose

Computação Distribuída - Nabor Das Chagas Mendonça

Jean de Souza Morais - 2323225

Thiago Cavalcante da Silva - 2320488

1. Introdução

O objetivo deste trabalho é executar múltiplas instâncias do WordPress para aumentar a escalabilidade e a disponibilidade da aplicação. Para isso, o Nginx foi configurado como balanceador de carga HTTP à frente de três contêineres WordPress, todos ligados a um mesmo banco MySQL. Toda a arquitetura, composta por cinco contêineres, foi descrita em um único arquivo docker-compose.yml.

Este relatório descreve, na ordem em que foram executados, o ambiente, os arquivos de configuração, o problema encontrado na primeira inicialização e os testes realizados. Duas questões recebem destaque: se o balanceamento é aleatório (não é: o Nginx usa por padrão o algoritmo Round Robin) e como isso foi comprovado por meio de um script PHP auxiliar, o quem.php, que revela qual contêiner respondeu a cada requisição.

2. Fundamentação

2.1 Docker e Docker Compose

O Docker executa aplicações em contêineres isolados a partir de imagens. O Docker Compose permite declarar vários contêineres, suas imagens, variáveis de ambiente, portas e volumes em um arquivo YAML e subi-los com um único comando. O Compose cria uma rede interna na qual os serviços se encontram pelo nome (por exemplo, mysql, wordpress1).

2.2 Nginx como balanceador de carga

O Nginx pode atuar como proxy reverso: recebe as requisições dos clientes e as repassa a um grupo de servidores definido no bloco upstream, por meio da diretiva proxy_pass. Distribuir requisições entre várias instâncias melhora o aproveitamento de recursos, reduz a latência e permite tolerar a falha de parte dos servidores.

2.3 Algoritmo Round Robin

Quando o bloco upstream não especifica nenhum método de balanceamento, o Nginx usa o Round Robin: as requisições são distribuídas em sequência circular, na ordem em que os servidores foram declarados. Cada servidor tem peso 1 por padrão (parâmetro weight), de modo que todos recebem a mesma quantidade de requisições. Não há sorteio e a ordem é previsível.

Se uma tentativa de conexão a um servidor falha, o Nginx repassa a mesma requisição ao próximo da lista (failover) e passa a evitar o servidor com problema por um período. Pelos valores padrão (max_fails=1 e fail_timeout=10s), isso dura cerca de 10 segundos.

3. Ambiente e instalação

O trabalho foi realizado em um computador com Windows 10 Pro versão 22H2 (64 bits) e virtualização habilitada. Como nada estava instalado, os passos foram:

1. **Verificação do sistema:** versão do Windows (comando winver) e virtualização habilitada (Gerenciador de Tarefas, aba Desempenho, CPU).
2. **WSL 2:** no PowerShell como administrador, executou-se wsl --install --no-distribution, seguido de reinício do computador.
3. **Docker Desktop:** instalado com a opção de usar o WSL 2 como backend.
4. **Verificação:** docker --version (Docker 29.8.2) e docker run hello-world, que exibiu a mensagem Hello from Docker!
5. **Trabalho 1 (pré-requisito):** foram baixadas a imagem mysql:5.7 e a imagem do WordPress; os contêineres wordpress-db (porta 3306) e wordpress (porta 8080) foram executados com --link, e a tela de instalação do WordPress apareceu em http://localhost:8080.
6. **Preparação para o Trabalho 2:** os contêineres do Trabalho 1 foram parados com docker stop wordpress wordpress-db, para liberar as portas. A versão do Compose foi confirmada com docker compose version (v5.6.0). Nas versões atuais o comando é docker compose, com espaço, mas o arquivo continua se chamando docker-compose.yml.

4. Implementação

4.1 Preparação do projeto

Foi criada a pasta do projeto, wordpress-cluster, no diretório do usuário, e nela o arquivo nginx.conf, criado no VS Code.

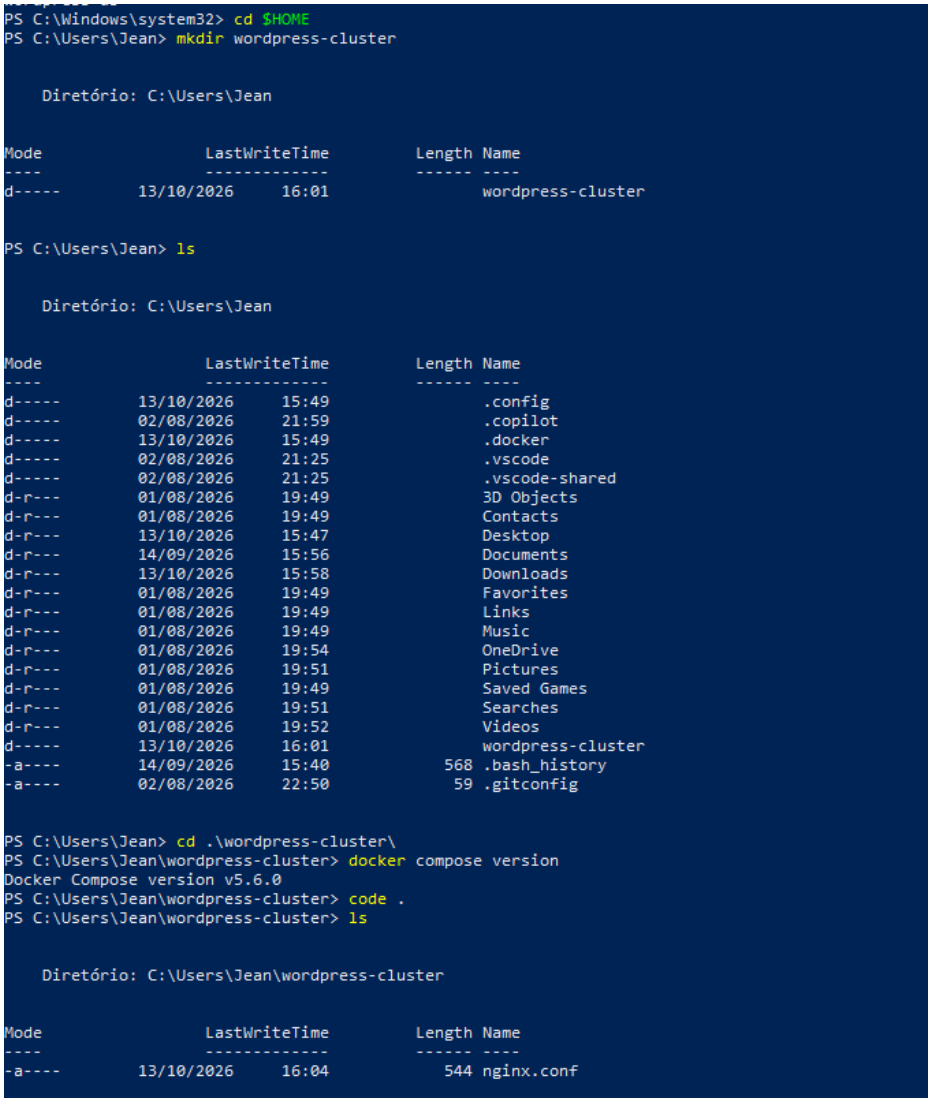


Figura 1. Criação da pasta wordpress-cluster, verificação do Docker Compose (v5.6.0) e arquivo nginx.conf.

4.2 Configuração do Nginx (nginx.conf)

```
events { worker_connections 1024; }
http {
  upstream wordpress {
    server wordpress1;
    server wordpress2;
    server wordpress3;
  }
  server {
    listen 80 default_server;
    listen [::]:80 default_server;
    root /usr/share/nginx/html;
    index index.php;
    location / {
```

```
      add_header X-Upstream $upstream_addr;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header x-forwarded-for $proxy_add_x_forwarded_for;
      proxy_pass http://wordpress;
    }
  }
}
```

O bloco upstream wordpress define o pool com os três serviços; os nomes precisam ser idênticos aos nomes dos serviços no docker-compose.yml. Como nenhum método é indicado, vale o Round Robin com pesos iguais. A diretiva add_header X-Upstream $upstream_addr inclui na resposta o endereço IP do contêiner que atendeu a requisição, o que permite observar o balanceamento de fora. A diretiva proxy_pass repassa as requisições ao pool.

4.3 Arquivo docker-compose.yml

```
services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: r00t
      MYSQL_DATABASE: wordpress
      MYSQL_USER: usr-wordpress
      MYSQL_PASSWORD: pwd-wordpress

  wordpress1:
    image: wordpress:5.4.2-php7.2-apache
    depends_on:
      - mysql
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_USER: usr-wordpress
      WORDPRESS_DB_PASSWORD: pwd-wordpress
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - ./wordpress:/var/www/html

  wordpress2:
    image: wordpress:5.4.2-php7.2-apache
    depends_on:
      - mysql
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_USER: usr-wordpress
      WORDPRESS_DB_PASSWORD: pwd-wordpress
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - ./wordpress:/var/www/html

  wordpress3:
    image: wordpress:5.4.2-php7.2-apache
    depends_on:
      - mysql
    environment:
```

```
    WORDPRESS_DB_HOST: mysql:3306
    WORDPRESS_DB_USER: usr-wordpress
    WORDPRESS_DB_PASSWORD: pwd-wordpress
    WORDPRESS_DB_NAME: wordpress
  volumes:
    - ./wordpress:/var/www/html

nginx:
  image: nginx:1.19.0
  depends_on:
    - wordpress1
    - wordpress2
    - wordpress3
  ports:
    - "80:80"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
    - ./wordpress:/usr/share/nginx/html
```

As decisões de projeto foram:

- **Imagens exigidas pelo enunciado:** mysql:5.7, wordpress:5.4.2-php7.2-apache e nginx:1.19.0.
- **Mesmo banco para os três WordPress:** todos usam WORDPRESS_DB_HOST: mysql:3306, de modo que produzem o mesmo conteúdo.
- **Sem ports nos WordPress e no MySQL:** assim eles não são acessíveis pela máquina hospedeira. O único serviço com porta publicada é o nginx (80:80).
- **Pasta compartilhada:** ./wordpress é montada em /var/www/html nos três WordPress e em /usr/share/nginx/html no Nginx, como pedido no enunciado.
- **nginx.conf montado em /etc/nginx/nginx.conf**, substituindo a configuração padrão da imagem.

4.4 Validação da sintaxe

Antes de subir os contêineres, o arquivo foi validado com docker compose config, que exibe a configuração interpretada. O resultado mostrou os cinco serviços, as imagens corretas, os volumes e apenas a porta 80 publicada no nginx.

```
PS C:\Users\Jean\wordpress-cluster> docker compose config
name: wordpress-cluster
services:
  mysql:
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_PASSWORD: pwd-wordpress
      MYSQL_ROOT_PASSWORD: r00t
      MYSQL_USER: usr-wordpress
    image: mysql:5.7
    networks:
      default: null
  nginx:
    image: nginx:1.19.0
    networks:
      default: null
    volumes:
      - type: bind
        source: C:\Users\Jean\wordpress-cluster\nginx.conf
        target: /etc/nginx/nginx.conf
        bind: {}
      - type: bind
        source: C:\Users\Jean\wordpress-cluster\wordpress
        target: /usr/share/nginx/html
        bind: {}
    depends_on:
      wordpress1:
        condition: service_started
        required: true
      wordpress2:
        condition: service_started
        required: true
      wordpress3:
        condition: service_started
        required: true
    ports:
      - mode: ingress
        target: 80
        published: "80"
        protocol: tcp
  wordpress1:
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_NAME: wordpress
      WORDPRESS_DB_PASSWORD: pwd-wordpress
      WORDPRESS_DB_USER: usr-wordpress
    image: wordpress:5.4.2-php7.2-apache
    networks:
      default: null
    volumes:
      - type: bind
        source: C:\Users\Jean\wordpress-cluster\wordpress
        target: /var/www/html
        bind: {}
    depends_on:
      mysql:
        condition: service_started
        required: true
```

Figura 2. docker compose config (parte 1): serviços mysql, nginx e wordpress1. Só o nginx publica a porta 80.

```
      required: true
  wordpress2:
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_NAME: wordpress
      WORDPRESS_DB_PASSWORD: pwd-wordpress
      WORDPRESS_DB_USER: usr-wordpress
    image: wordpress:5.4.2-php7.2-apache
    networks:
      default: null
    volumes:
      - type: bind
        source: C:\Users\Jean\wordpress-cluster\wordpress
        target: /var/www/html
        bind: {}
    depends_on:
      mysql:
        condition: service_started
        required: true
  wordpress3:
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_NAME: wordpress
      WORDPRESS_DB_PASSWORD: pwd-wordpress
      WORDPRESS_DB_USER: usr-wordpress
    image: wordpress:5.4.2-php7.2-apache
    networks:
      default: null
    volumes:
      - type: bind
        source: C:\Users\Jean\wordpress-cluster\wordpress
        target: /var/www/html
        bind: {}
    depends_on:
      mysql:
        condition: service_started
        required: true
networks:
  default:
    name: wordpress-cluster_default
```

Figura 3. docker compose config (parte 2): wordpress2 e wordpress3 com o mesmo banco e a mesma pasta compartilhada.

4.5 Subida dos contêineres e primeiro problema

Os contêineres foram iniciados com docker compose up -d. O docker compose ps mostrou os cinco em execução, e o Docker Desktop passou a exibir o projeto wordpress-cluster com os serviços mysql, wordpress1, wordpress2, wordpress3 e nginx.

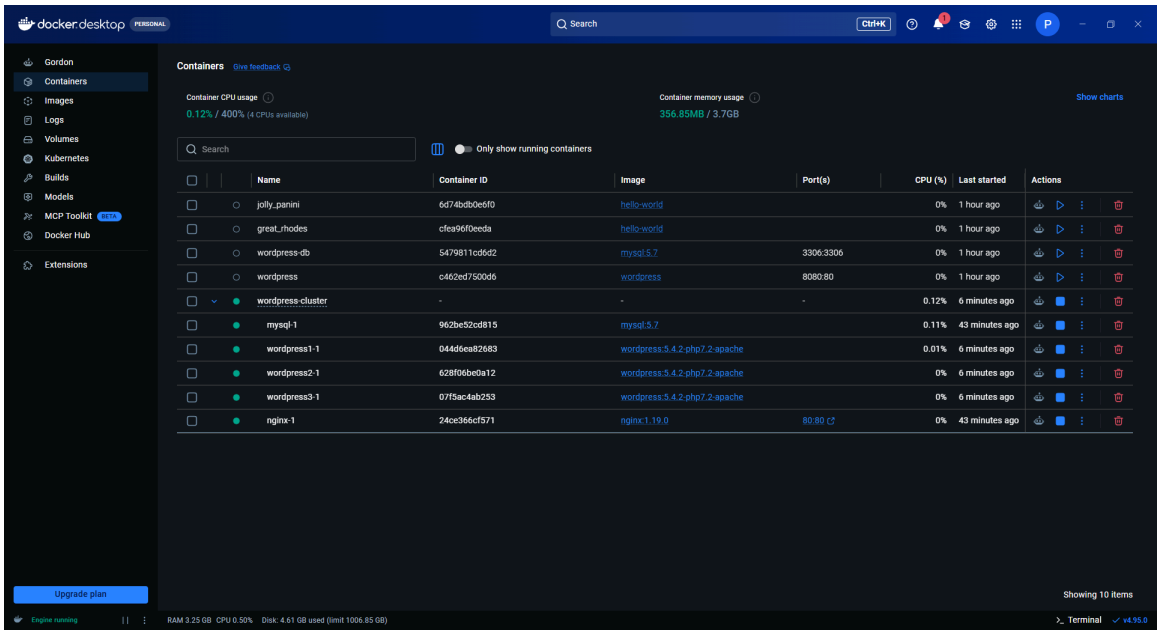


Figura 4. Docker Desktop com o projeto wordpress-cluster: mysql, três WordPress e nginx em execução. Os contêineres em cinza são do Trabalho 1 e estão parados.

Nos primeiros testes com curl, porém, o X-Upstream listava vários IPs numa mesma resposta (por exemplo 172.18.0.4:80, 172.18.0.5:80, 172.18.0.3:80) e uma das respostas foi 502 Bad Gateway. Isso indica que o Nginx tentou servidores que não responderam e passou ao seguinte. Os logs (docker compose logs --tail 15 wordpress1 wordpress2 wordpress3) apresentaram que na primeira inicialização, a pasta ./wordpress estava vazia e os três contêineres tentaram copiar os arquivos do WordPress para ela ao mesmo tempo. O wordpress3 concluiu a cópia, mas o wordpress1 e o wordpress2 falharam com mensagens do tar (Cannot open: File exists) e encerraram.

Como a pasta já estava preenchida, bastou iniciar os dois contêineres novamente, com docker compose start wordpress1 wordpress2. Depois disso, os três passaram a responder.

4.6 Instalação do WordPress

Com os três contêineres no ar, a instalação foi concluída pelo navegador, em http://localhost: escolha do idioma, título do site (Trabalho 2 Comp. Distri.) e criação do usuário administrador. A figura mostra o site já instalado, com a barra de administração visível.

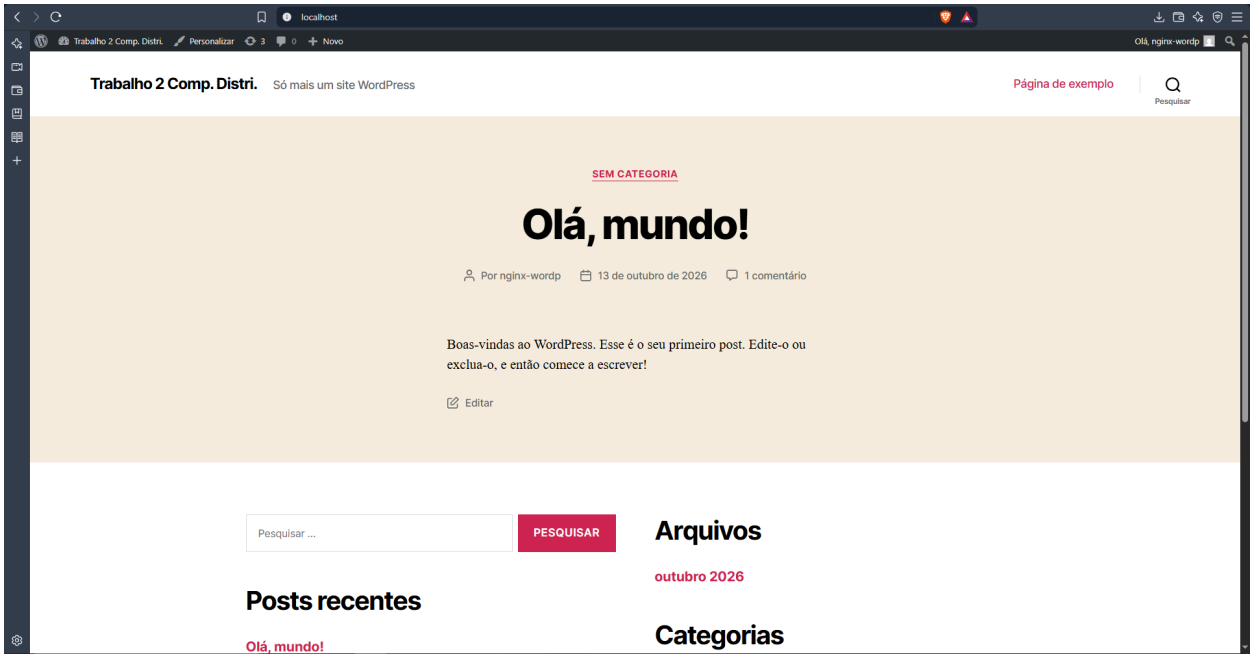


Figura 5. Site WordPress instalado, acessado por http://localhost (via Nginx), com a barra de administração visível.

4.7 O script quem.php

Para provar qual contêiner atende cada requisição, foi criado o arquivo quem.php dentro da pasta ./wordpress:

```
<?php
echo "Respondido pelo container: " . gethostname() . PHP_EOL;
```

No Docker, o hostname de um contêiner é, por padrão, o seu ID curto. Como a pasta é compartilhada, o mesmo arquivo é servido pelos três WordPress, mas cada um imprime o seu próprio ID. O ID é, portanto, a assinatura de quem respondeu. A associação entre IDs, nomes e IPs foi obtida com docker ps --format e docker inspect:

Contêiner	ID (hostname)	IP (antes do religamento)
wordpress-cluster-wordpress1-1	044d6ea82683	172.18.0.4
wordpress-cluster-wordpress2-1	628f06be0a12	172.18.0.5
wordpress-cluster-wordpress3-1	07f5ac4ab253	172.18.0.3

5. Testes e resultados

5.1 Teste proposto no enunciado: curl e X-Upstream

O enunciado sugere executar várias vezes curl -I http://localhost/ e observar o cabeçalho X-Upstream. No PowerShell foi usado curl.exe, porque curl sozinho é um apelido de outro comando. A resposta 302 Found é esperada, pois o WordPress redirecionava para a página de instalação. O que interessa é o X-Upstream, que mudou a cada requisição.

```
PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:19:05 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.3:80

PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:19:06 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.4:80

PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:19:08 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.5:80

PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:26:51 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.3:80

PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:26:52 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.4:80
```

Figura 6. curl.exe -I http://localhost/ (parte 1): o X-Upstream alterna entre 172.18.0.3, 172.18.0.4 e 172.18.0.5.

```
PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:26:54 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.5:80

PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:26:55 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.3:80

PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:26:56 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.4:80

PS C:\Users\Jean\wordpress-cluster> curl.exe -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Tue, 13 Oct 2026 19:26:57 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/7.2.33
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
X-Upstream: 172.18.0.5:80
```

Figura 7. curl.exe -I http://localhost/ (parte 2): a rotação continua em 172.18.0.5, .3, .4 e .5.

Nas seis requisições consecutivas das duas figuras, feitas em poucos segundos, o X-Upstream seguiu a sequência 172.18.0.3, 172.18.0.4, 172.18.0.5, 172.18.0.3, 172.18.0.4, 172.18.0.5, sem nenhum desvio. Pelo mapeamento da seção 4.7, isso corresponde a wordpress3, wordpress1, wordpress2, wordpress3, wordpress1, wordpress2, ou seja, a ordem circular dos servidores do upstream.

5.2 Round Robin

O Nginx não sorteia o servidor de cada requisição. Como o upstream não define método nem weight, ele aplica o Round Robin com pesos iguais: percorre os servidores em ordem circular, um por requisição. Se o balanceamento fosse aleatório, a sequência não teria padrão e, em poucas requisições, um contêiner poderia receber muito mais que os outros.

Para comprovar, foram feitas nove requisições seguidas ao quem.php, com o comando:

1..9 | ForEach-Object { curl.exe -s http://localhost/quem.php }.

O resultado foi:

Requisição	ID que respondeu	Contêiner
1, 4, 7	628f06be0a12	wordpress2
2, 5, 8	07f5ac4ab253	wordpress3
3, 6, 9	044d6ea82683	wordpress1

A sequência 628f, 07f5, 044d se repetiu três vezes sem variação, e cada contêiner atendeu exatamente 3 das 9 requisições. Esse é o comportamento determinístico do Round Robin. A mesma figura mostra, em seis requisições à página inicial, o X-Upstream mudando de IP e o título do site sendo sempre o mesmo, além do mapeamento entre IDs, nomes e IPs.

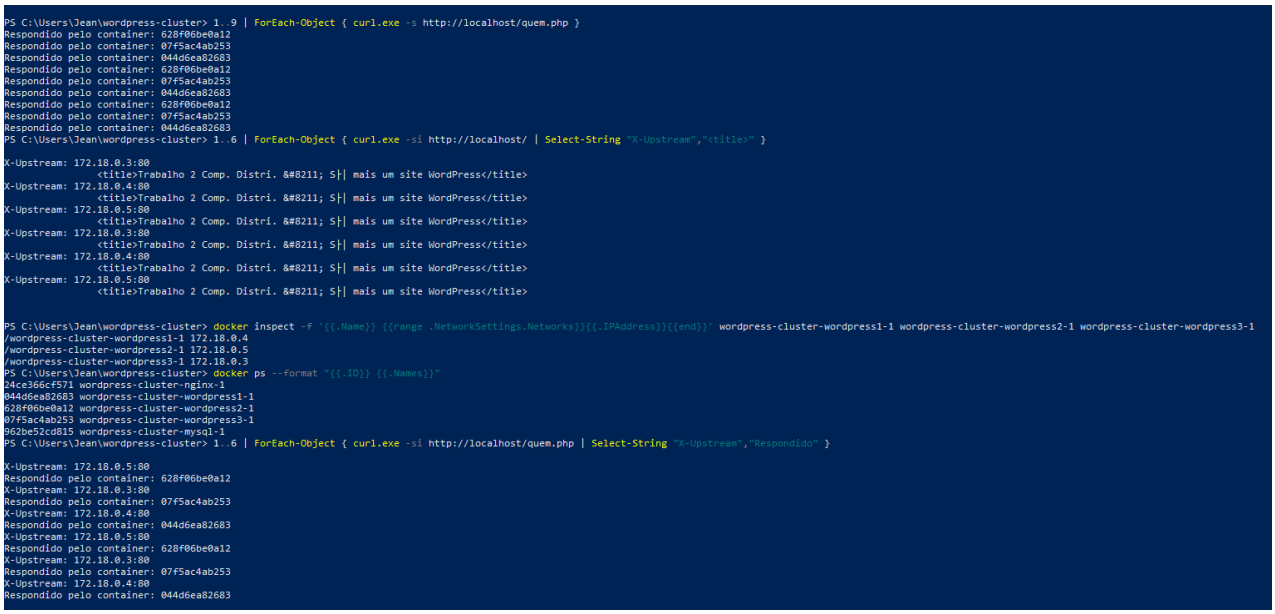


Figura 8. Nove requisições ao quem.php (cada contêiner responde 3 vezes), título igual nas respostas da página inicial e mapeamento entre ID, nome e IP dos contêineres.

5.3 Prova visual no navegador

A mesma verificação foi feita no navegador, em http://localhost/quem.php, atualizando a página (F5). Cada atualização exibiu o ID de um contêiner diferente. As três figuras a seguir mostram os três contêineres respondendo.

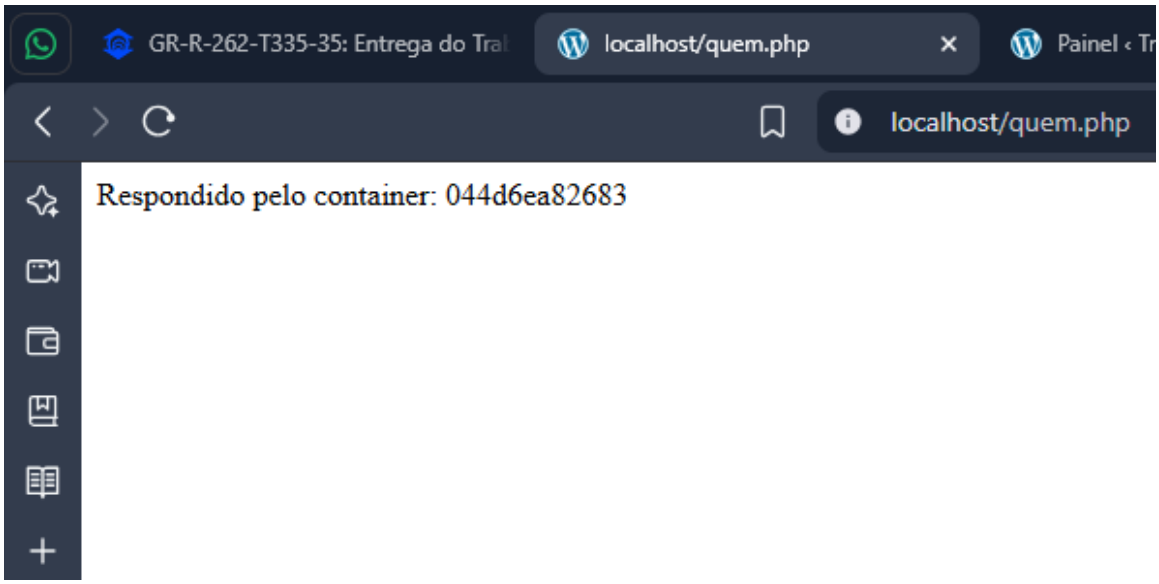


Figura 9. quem.php no navegador, respondido pelo wordpress1 (044d6ea82683).

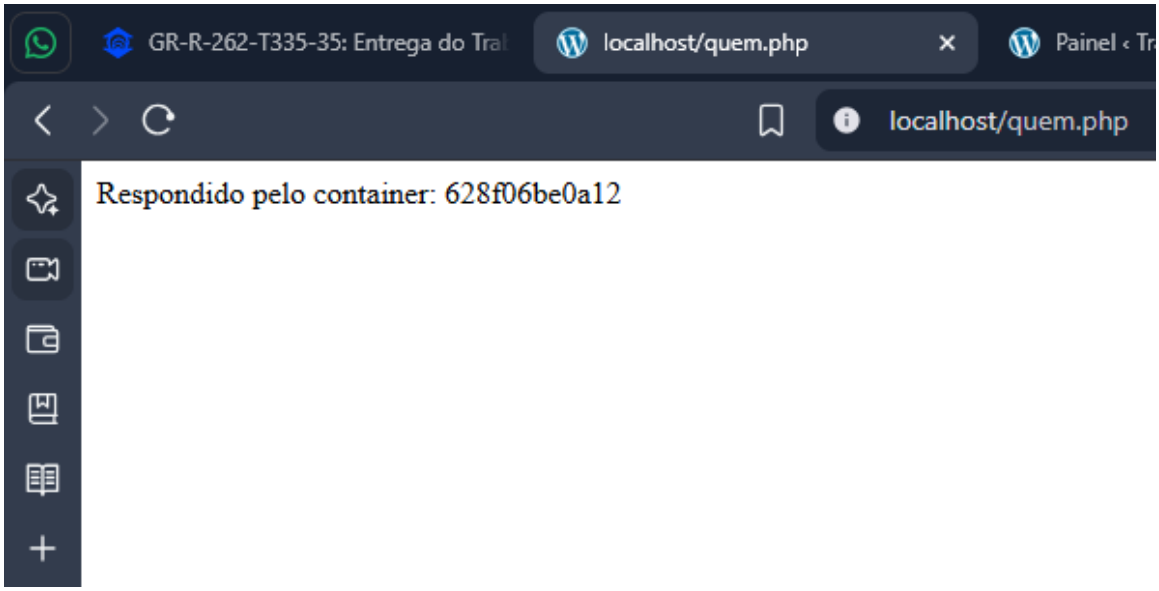


Figura 10. quem.php no navegador, respondido pelo wordpress2 (628f06be0a12).

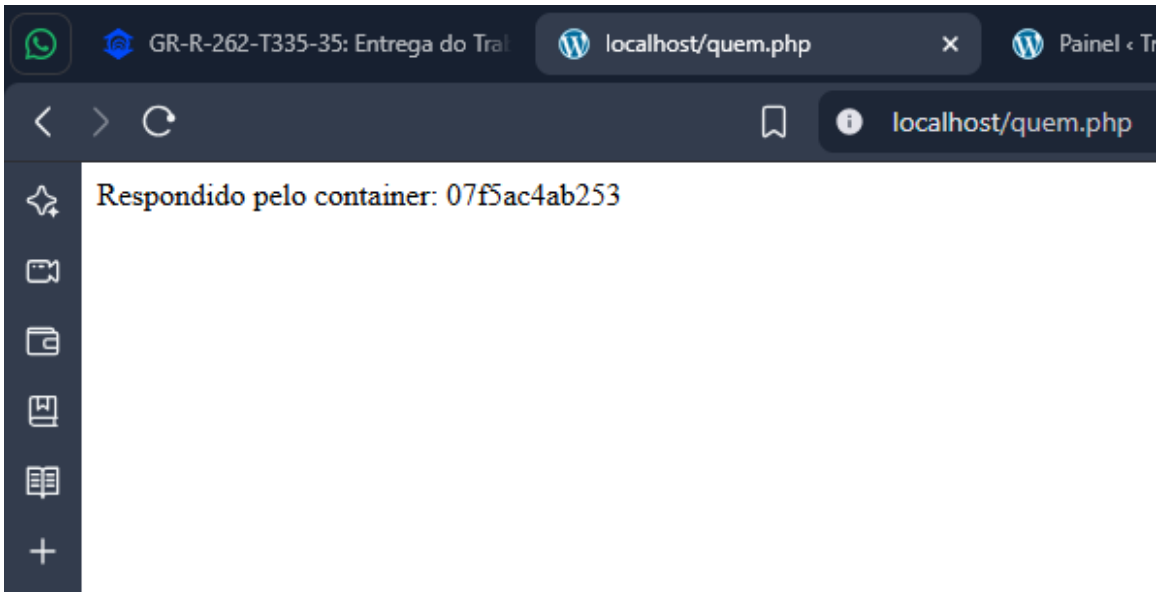


Figura 11. quem.php no navegador, respondido pelo wordpress3 (07f5ac4ab253).

Observação: ao usar o navegador, a ordem pode parecer irregular, porque ele também requisita o ícone da aba (favicon.ico), o que consome uma vez da rotação. Por isso a comprovação do padrão foi feita pelo terminal, onde há uma única requisição por vez.

5.4 Os três contêineres entregam o mesmo conteúdo

Apesar de os contêineres serem diferentes (IDs, nomes e IPs distintos), o conteúdo é o mesmo:

- Nas seis requisições à página inicial, com três contêineres diferentes respondendo, o título foi sempre Trabalho 2 Comp. Distri. (os caracteres estranhos que aparecem no terminal são só um problema de codificação do PowerShell).
- A sessão de administrador do WordPress (barra de administração visível na Figura 5) permaneceu válida enquanto as requisições passavam por contêineres diferentes.

Isso ocorre porque os três WordPress compartilham o mesmo banco MySQL, que guarda posts, usuários e configurações, e a mesma pasta /var/www/html, que contém o código e o wp-config.php.

5.5 Isolamento das portas e pasta compartilhada

O enunciado exige que os WordPress e o MySQL não sejam acessíveis diretamente pela máquina e que o Nginx enxergue a pasta compartilhada. Foram executados:

- docker compose ps: apenas o nginx aparece com porta publicada (0.0.0.0:80->80/tcp). O MySQL mostra somente 3306/tcp e 33060/tcp, e os WordPress somente 80/tcp, portas internas que não estão mapeadas para a máquina.
- Test-NetConnection localhost -Port 3306 -InformationLevel Quiet: retornou False, ou seja, a máquina não alcança o MySQL.
- docker compose exec nginx ls /usr/share/nginx/html: lista os arquivos do WordPress, incluindo o quem.php, comprovando que o Nginx vê a mesma pasta dos WordPress.

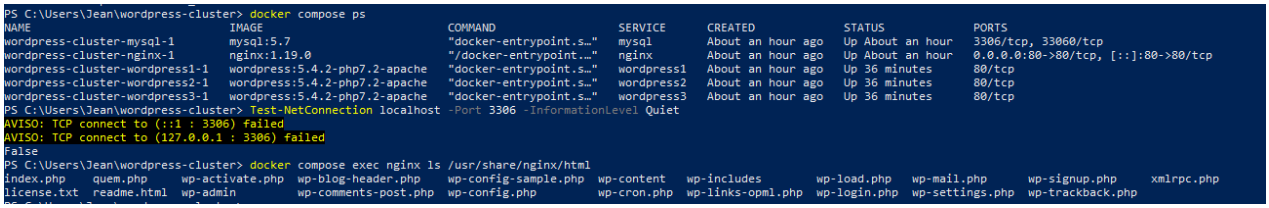


Figura 12. docker compose ps (só o nginx publica porta), Test-NetConnection na porta 3306 (False) e ls em /usr/share/nginx/html (arquivos compartilhados, com o quem.php).

5.6 Tolerância a falhas

Para verificar o failover, os contêineres WordPress foram parados um a um, sem religar nenhum entre os testes, e o quem.php foi consultado seis vezes a cada etapa com o comando *1..6 | ForEach-Object { curl.exe -si http://localhost/quem.php | Select-String "X-Upstream","Respondido" }*

Teste 1: wordpress1 parado. Apenas wordpress2 (628f06be0a12) e wordpress3 (07f5ac4ab253) responderam, três vezes cada, sem erro para o cliente. Numa das requisições o X-Upstream listou dois IPs (172.18.0.4:80, 172.18.0.5:80): o Nginx tentou o wordpress1, parado, e repassou a requisição ao wordpress2.

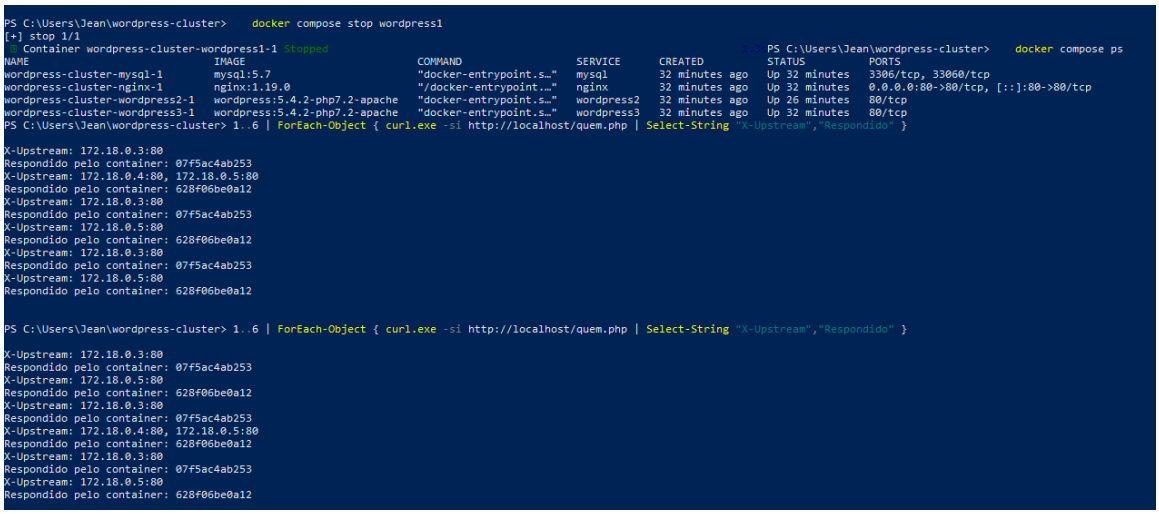


Figura 13. Teste 1: wordpress1 parado. Apenas wordpress2 e wordpress3 respondem; uma resposta mostra dois IPs no X-Upstream (failover).

Teste 2: wordpress1 e wordpress2 parados. Todas as seis respostas vieram do wordpress3 (07f5ac4ab253). O X-Upstream listou dois IPs em duas requisições (172.18.0.5:80, 172.18.0.3:80 e 172.18.0.4:80, 172.18.0.3:80), uma para cada servidor parado: em cada caso o Nginx tentou o servidor fora do ar e passou ao wordpress3. O site continuou disponível com um único contêiner saudável.

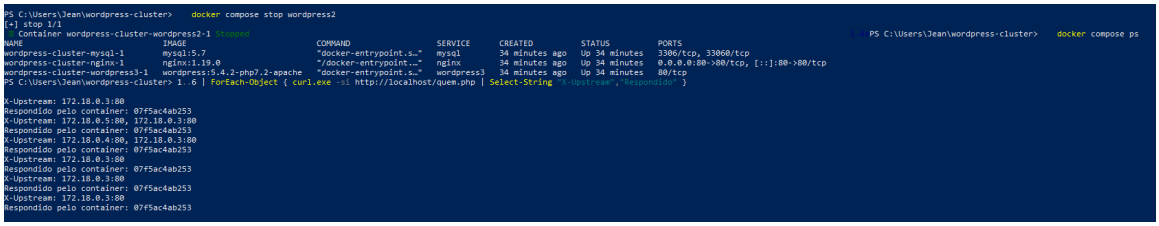


Figura 14. Teste 2: wordpress1 e wordpress2 parados. Todas as respostas vêm do wordpress3.

Teste 3: os três WordPress parados. Restaram em execução apenas o nginx e o mysql. As três requisições retornaram o código 502 (Bad Gateway): o Nginx continuou no ar, mas não havia nenhum servidor para receber a requisição.



Figura 15. Teste 3: os três WordPress parados. Restam nginx e mysql, e as três requisições retornam 502.

Teste 4: contêineres religados. Com docker compose start wordpress1 wordpress2 wordpress3, os três voltaram a responder em rotação (044d6ea82683, 628f06be0a12, 07f5ac4ab253), como no início.

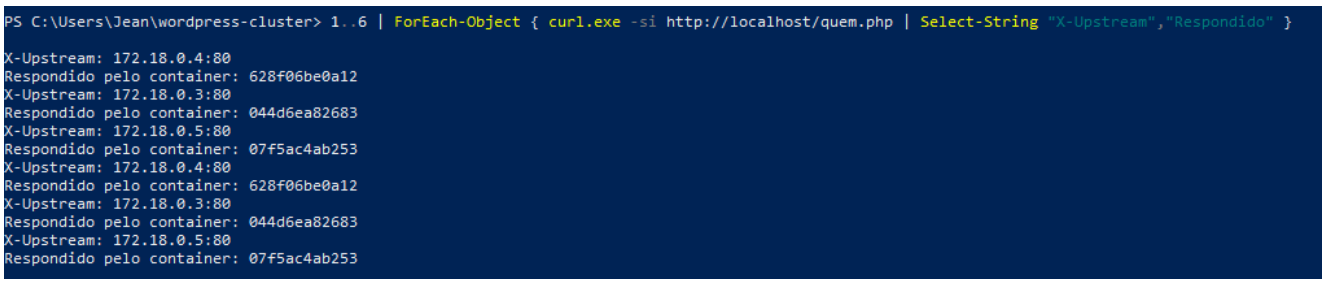


Figura 16. Teste 4: contêineres religados e rotação restabelecida entre os três WordPress.

Resumo dos testes:

Cenário	Quem respondeu	Resultado
Os 3 ligados	wordpress1, 2 e 3, em rotação	Normal
wordpress1 parado	wordpress2 e wordpress3	Sem erro para o cliente
wordpress1 e wordpress2 parados	wordpress3	6 de 6 respostas
Os 3 parados	Ninguém	502 Bad Gateway
Os 3 religados	wordpress1, 2 e 3	Rotação restabelecida

Detalhe observado no religamento. Primeiro, os IPs dos contêineres mudaram: no último teste, 172.18.0.3 passou a ser o wordpress1 (044d6ea82683), 172.18.0.4 o wordpress2 (628f06be0a12) e 172.18.0.5 o wordpress3 (07f5ac4ab253). Por isso a tabela da seção 4.7 vale apenas para os testes anteriores ao religamento, e o ID do contêiner é o identificador confiável.

6. Discussão e limitações

- **Resolução de nomes:** o Nginx de código aberto resolve os nomes wordpress1, wordpress2 e wordpress3 quando inicia. Se algum contêiner receber um IP fora do conjunto original, seria necessário reiniciar o Nginx (docker compose restart nginx). Neste trabalho os três IPs foram apenas reaproveitados em outra ordem, e o balanceamento continuou normal.
- **Primeira inicialização:** três contêineres copiando os arquivos do WordPress para a mesma pasta vazia ao mesmo tempo geram uma disputa. A solução usada foi iniciar de novo os contêineres que falharam. Uma alternativa seria subir primeiro um só contêiner e os demais depois.
- **Pontos únicos de falha:** o MySQL e o Nginx continuam sendo instância única. O balanceamento protege contra a falha de um WordPress, mas não do banco nem do próprio balanceador. A pasta compartilhada também é única, no disco da máquina.
- **Remoção dos contêineres:** o docker compose down remove os contêineres, e com eles os dados do MySQL (o site e o usuário criados), pois o banco não foi configurado com volume. Os arquivos da pasta ./wordpress permanecem no disco.

7. Conclusão

O trabalho atingiu seu objetivo: cinco contêineres foram descritos em um único docker-compose.yml, com o Nginx balanceando a carga entre três instâncias do WordPress ligadas ao mesmo MySQL. O Nginx aplica o Round Robin com pesos iguais, o que foi comprovado pelo quem.php, com cada contêiner atendendo 3 de 9 requisições na mesma ordem, e pelo cabeçalho X-Upstream. Os testes de desligamento mostraram o failover em funcionamento: o site continuou disponível com dois e com um contêiner, e só deixou de responder quando os três WordPress ficaram indisponíveis. Os requisitos de isolamento de portas e de pasta compartilhada também foram verificados.

Referências

- Docker. Docker Compose. https://docs.docker.com/compose/
- Docker Hub. Imagem oficial do WordPress. https://hub.docker.com/_/wordpress
- Docker Hub. Imagem oficial do MySQL. https://hub.docker.com/_/mysql
- Docker Hub. Imagem oficial do Nginx. https://hub.docker.com/_/nginx